

Đề Xuất Đồ Án Cơ Sở Dữ Liệu Phân Tán

1. Thông tin đề án

- Sinh viên thực hiện:** Hồ Ngọc Hoàng Anh
- Mã số sinh viên:** N23DCCN071
- Tên đề tài:** Merkle Tree Log Integrity: "Immutable Audit Trail" (Phát Hiện Và Truy Vết Lỗi Toàn Vẹn Nhật Ký Giao Dịch Bằng Cây Merkle Phân Tán)

2. Mục tiêu & Bài toán cần giải quyết

- Lý do chọn đề tài:** Trong các hệ thống cơ sở dữ liệu ngân hàng phân tán, nhật ký giao dịch (audit trail) phải được đảm bảo tính bất biến để phòng chống gian lận. Tuy nhiên, một mối đe dọa thường trực là các cuộc tấn công từ nội bộ (**Insider Attacks**), nơi quản trị viên cơ sở dữ liệu (Rogue DBA) tại các chi nhánh (Site B) có quyền truy cập trực tiếp vào máy chủ vật lý để sửa đổi hoặc xóa dữ liệu giao dịch bằng câu lệnh SQL thô, qua mặt hoàn toàn các lớp kiểm soát của ứng dụng. Đề án này giải quyết bài toán kiểm tra tính toàn vẹn của nhật ký giao dịch mà không cần truyền tải toàn bộ dữ liệu qua mạng, giúp phát hiện tức thời và chỉ điểm chính xác giao dịch bị thay đổi với chi phí cực thấp.
- Thuật toán cốt lõi:** Hiện thực hóa cấu trúc dữ liệu **Cây Merkle (Merkle Tree / Hash Tree)**. Cứ mỗi 100 dòng nhật ký giao dịch ngân hàng sẽ được gom lại thành một Block. Các nút lá chứa mã băm SHA-256 của từng giao dịch đơn lẻ. Các nút cha là mã băm kết hợp của hai nút con trực tiếp. **Root Hash** (Mã băm gốc) đại diện cho cả Block sẽ được đăng ký lưu trữ tại một bên thứ ba trung lập tin cậy (**TTP**). Khi thực hiện kiểm toán, hệ thống sẽ tính toán lại Root Hash từ CSDL hiện tại và đối chiếu với Root Hash gốc tại TTP. Nếu có sai lệch, hệ thống sẽ so sánh Leaf Hash đối chứng giữa bản sạch (Site A) và bản bị tấn công (Site B) để chỉ điểm chính xác ID giao dịch bị can thiệp.

3. Đặc tả Dataset (Tập dữ liệu)

- Nguồn dữ liệu:** Bộ sinh giao dịch ngân hàng giả lập tự động (scripts/generator.py).
- Quy mô:** Khởi tạo 500 bản ghi giao dịch (gồm 5 Block, mỗi block đúng 100 giao dịch) và có thể phát sinh thêm qua API.
- Schema (Lược đồ dữ liệu):**
 - TransactionID (TEXT, PRIMARY KEY, định dạng: TX-1XXXXX) - Mã giao dịch.
 - From_Account (TEXT) - Tài khoản chuyển tiền.
 - To_Account (TEXT) - Tài khoản nhận tiền.
 - Amount (REAL) - Số tiền giao dịch (USD).
 - Timestamp (TEXT, định dạng ISO-8601) - Thời gian giao dịch.
 - BlockID (INTEGER) - ID của Block chứa giao dịch.
- Chiến lược phân mảnh và nhân bản:** Mô phỏng nhân bản toàn phần ngang hàng (ROWA - Read-One/Write-All). Các giao dịch được điều phối nhân bản đồng bộ từ Coordinator sang hai phân mảnh cơ sở dữ liệu độc lập: Site A (bản sao sạch đối chứng) và Site B (bản sao chỉ nhánh, mục tiêu tấn công).

4. Kiến trúc hệ thống

- Các Node mạng:**
 - Coordinator (Port 5000):** Bộ điều phối trung tâm. Nhận giao dịch từ client, thực hiện nhân bản đồng bộ, tự động đóng block để dựng cây băm và cung cấp giao diện Dashboard điều khiển.

- ii. Site A (Port 5001): Phân mảnh lưu trữ bản sao sạch (Trusted Replica) để làm cơ sở đối chứng truy vết số.
 - iii. Site B (Port 5002): Phân mảnh chi nhánh (Vulnerable Replica), nơi giả lập cuộc tấn công can thiệp SQL vật lý trực tiếp từ DBA.
 - iv. Trusted Third Party - TTP (Port 5003): Bên thứ ba độc lập lưu trữ Block ID và các Root Hash bất biến để phục vụ kiểm toán chéo.
- **Giao thức truyền thông:** HTTP/REST APIs sử dụng JSON payload.
 - **Công nghệ lưu trữ:** Mỗi Node quản lý một CSDL SQLite độc lập (site_a.db, site_b.db, và ttp.db).

5. Công nghệ & Kế hoạch triển khai

- **Ngôn ngữ lập trình:** Python 3.10
- **Triển khai:** Chạy các tiến trình microservices trên Localhost thông qua script điều phối (run_all.py).
- **Thư viện chính:** Flask (REST APIs), SQLite3 (CSDL), Chart.js (vẽ đồ thị hiệu năng), Vanilla HTML/CSS/JS (Giao diện Web Dashboard phong cách Glassmorphism).

6. Tiêu chí đánh giá & Phân tích thực nghiệm

- **Chỉ số định lượng:**
 - i. **Build Time (ms):** Thời gian dựng cây Merkle từ danh sách giao dịch.
 - ii. **Audit Verification Time (ms):** Thời gian đối chiếu mã băm phát hiện lỗi.
 - iii. **Storage Overhead (%):** Tỷ lệ dung lượng siêu dữ liệu Root Hash tại TTP so với dung lượng lưu trữ gốc.
- **Kịch bản lỗi (The "Failure" Scenario):** Mô phỏng cuộc tấn công **Insider Attack** bằng cách chạy truy vấn SQL thô thay đổi số tiền giao dịch TX-100150 tại Site B từ \$100.00 thành \$1000.00. Khi chạy Detector, hệ thống phải phát cảnh báo TAMPERED tại Block 2, đồng thời định vị chính xác mã giao dịch bị sửa đổi, số tiền ban đầu tại Site A và số tiền giả mạo tại Site B.

7. Mốc thời gian dự án

- **Mốc 1 (13/04 - 30/04): Nghiên cứu và Thiết lập môi trường**
 - Tìm hiểu cơ sở lý thuyết về cấu trúc cây Merkle, mã băm SHA-256 và cơ chế nhân bản dữ liệu phân tán (ROWA).
 - Thiết lập môi trường CSDL SQLite vật lý riêng biệt cho Site A, Site B và TTP.
 - Xây dựng script tự động sinh dữ liệu giao dịch mẫu (generator.py).
- **Mốc 2 (01/05 - 20/05): Phát triển thuật toán & Lập trình mạng phân tán**
 - Hiện thực hóa cấu trúc dữ liệu Merkle Tree thủ công (core/merkle.py), tối ưu hóa việc băm và xử lý phần tử lẻ.
 - Xây dựng 4 node microservices REST API độc lập bằng Python Flask và kết nối truyền thông phân tán qua HTTP.
 - Triển khai cơ chế nhân bản đồng bộ Eager Replication theo nguyên tắc ROWA trên node Coordinator.
- **Mốc 3 (21/05 - 02/06): Tích hợp giao diện, Giả lập tấn công, Kiểm toán & Hoàn thiện**
 - Thiết kế giao diện Web Dashboard trực quan hóa cấu trúc cây Merkle nhị phân và vẽ biểu đồ hiệu năng Chart.js.
 - Lập trình bộ giả lập tấn công (attack.py) và viết script kiểm toán điều tra số (detector.py).
 - Thực nghiệm giả lập các kịch bản sập node (Site B sập, TTP sập) để đánh giá khả năng chịu lỗi và tính nhất quán.
 - Quay video demo 3-5 phút và hoàn thiện toàn bộ báo cáo đồ án nộp ngày 02/06.